Truck-N-Trailer Autonomous Parking System

ME 235 Final Project Report

Eric Chuang • Evan Grealish • Aryan Kulkarni • Xiao Liu

May 12, 2026

1 Background and Motivation

Backing up a truck with a trailer is one of the most challenging things to do behind the wheel. It is a nonholonomic problem and you must ensure the system does not jackknife. It takes experienced drivers years to develop the skills for this control action. Because of this, it is an interesting control problem that is explored in this project.

This project builds a small physical model of the truck and trailer system with motors controlled by an ESP32 board, and integrates overhead vision and a laptop-hosted route planning stack. Our goal is to simulate a real-world situation where a driver needs to park a truck and trailer system, and could navigate into place manually or with an auto parking feature.

2 Description of GUI, Real-Time, and Multitasking Implementation

2.1 Graphical User Interface

The interface is organized around two main modes:

- **Manual Mode:** Provides forward, back, left, and right control buttons. A speed slider lets the user set the target RPM, and the GUI sends wheel commands directly over UART to the ESP32.
- **Automatic Mode:** Engages the MPC controller. A Start/Stop button initiates the autonomous parking sequence. The GUI feeds the current state from computer vision into the planner, then sends the resulting wheel speed commands to the firmware.

As shown in Figure 1, the GUI also displays a live overhead camera view drawn via OpenCV, including bounding boxes for the truck and trailer, the defined parking spot, and the planned trajectory from the MPC in Automatic Mode. A simulated dial widget shows the current hitch angle: the left dial is pulled from the potentiometer reading off the ESP32, and the right dial is computed from the ArUco marker positions detected by the camera. Telemetry cards show actual wheel RPMs, commanded speeds, and connection status in real time.

Before autonomous operation can begin, the potentiometer must be calibrated to map raw ADC counts to physical hitch angle in degrees. The GUI provides a step-by-step calibration wizard: the user positions the hitch at four known angles (maximum negative, zero, maximum positive, and zero again to capture hysteresis), captures a measurement at each step, and the system fits a piecewise-linear mapping. This calibration is a prerequisite for accurate hitch-angle feedback in both the dashboard display and the MPC state estimator.

[Insert GUI screenshot here — upload `gui_screenshot.png` *to Overleaf]*

Figure 1: GUI of Truck-n-Trailer (Automatic Mode, MPC running).

2.2 Real-Time Implementation

Real-time behavior in this system spans two computing layers with different timing requirements and different notions of "real-time."

Firmware — hard real-time. The ESP32 runs motor PID control at a fixed 100 Hz loop rate. This is hard real-time: the PID algorithm requires a consistent Δt to produce stable control output, and the GPTimer ISR enforces this with hardware-level precision independent of task scheduling. Encoder telemetry (motor RPMs and potentiometer reading) is sent to the laptop at 10 Hz. The 100 Hz rate was chosen to be fast enough to track velocity targets accurately across the full RPM range while leaving sufficient headroom for the lower-priority communication task.

Laptop — soft real-time. The laptop side runs two `QTimer` callbacks within a single PyQt6 event loop:

- **Camera timer (30 Hz, 33 ms interval):** Each tick grabs a frame from the overhead camera (`cv2.VideoCapture.read()`, blocking I/O) then runs ArUco marker detection (`detectMarkers()`, typically 10–25 ms depending on frame content and marker visibility). When three markers are found (truck ID 0, trailer ID 1, goal ID 10), the pipeline computes world-frame poses and outputs the vehicle state vector `vision_q` and goal position `goal_xy`. Effective throughput is therefore 10–30 fps in practice.
- **Send timer (10 Hz, 100 ms interval):** Polls the UART receive buffer for the latest telemetry, runs the MPC solver if in autonomous mode, and writes the resulting RPM targets back over UART. The MPC solve is the dominant cost in this callback: IPOPT typically converges in 50–200 ms with a warm start from the previous solution. Because this runs synchronously in the Qt event loop, a slow solve stretches the 100 ms send interval, introducing jitter in the motor commands. The system tolerates occasional late commands without failure, but tracking quality degrades — a soft real-time constraint.

All data and command communication between the laptop and firmware occurs over UART at 115 200 baud.

Table 1: Timing summary across system layers.

Component	Rate	Deadline type	Dominant cost
ESP32 PID loop	100 Hz	Hard	GPTimer ISR latency (<1 µs)
ESP32 telemetry TX	10 Hz	Soft	UART byte write (∼1 ms)
Laptop camera tick	30 Hz	Soft	ArUco detection (10–25 ms)
Laptop send tick	10 Hz	Soft	MPC solve (50–200 ms)

2.3 Multitasking Architecture

Firmware — FreeRTOS. The firmware runs two concurrent FreeRTOS tasks with different priorities:

- **Control Task (higher priority):** Triggered at 100 Hz by the GPTimer ISR via `vTaskNotifyGiveFromISR`. Reads encoder deltas, converts counts to RPM, runs independent PID controllers for the left and right wheels, applies a static feedforward term, and writes PWM outputs to the motors. Also handles the kill-switch state and sends telemetry at 10 Hz.

- **Communication Task (lower priority):** Triggered by the UART receive interrupt when a full command packet arrives. Parses incoming target RPM values and writes them to shared state protected by a FreeRTOS mutex. Also reads the potentiometer ADC value and forwards it to the laptop in the telemetry stream.

The shared state between the two tasks is protected through FreeRTOS synchronization primitives. If the command is stale beyond a configurable timeout — indicating the laptop has stopped sending — the Control Task automatically falls back to zero RPM as a safety measure.

The interrupt structure includes: (1) hardware interrupts on four encoder channels (Channel A and B for each motor) to count quadrature encoder pulses; (2) an interrupt for the emergency stop button that bypasses the normal control loop immediately; and (3) the GPTimer alarm ISR that unblocks the Control Task at exactly 100 Hz.

Laptop — Qt event loop. The laptop side uses a deliberate single-threaded design. Both timers (camera at 30 Hz and send at 10 Hz) are dispatched sequentially by the Qt event loop — they never execute concurrently. This means the shared vision state variables (`vision_q`, `goal_xy`) can be read by the send callback without locks, because the camera callback cannot interrupt it. The tradeoff is that a blocking MPC solve prevents the camera callback from firing until it finishes, potentially dropping camera frames during long solves. This was an acceptable tradeoff given the project scope; a production system would move the MPC solve to a background thread.

System architecture diagram. Figure 2 shows the complete data-flow architecture across both layers.

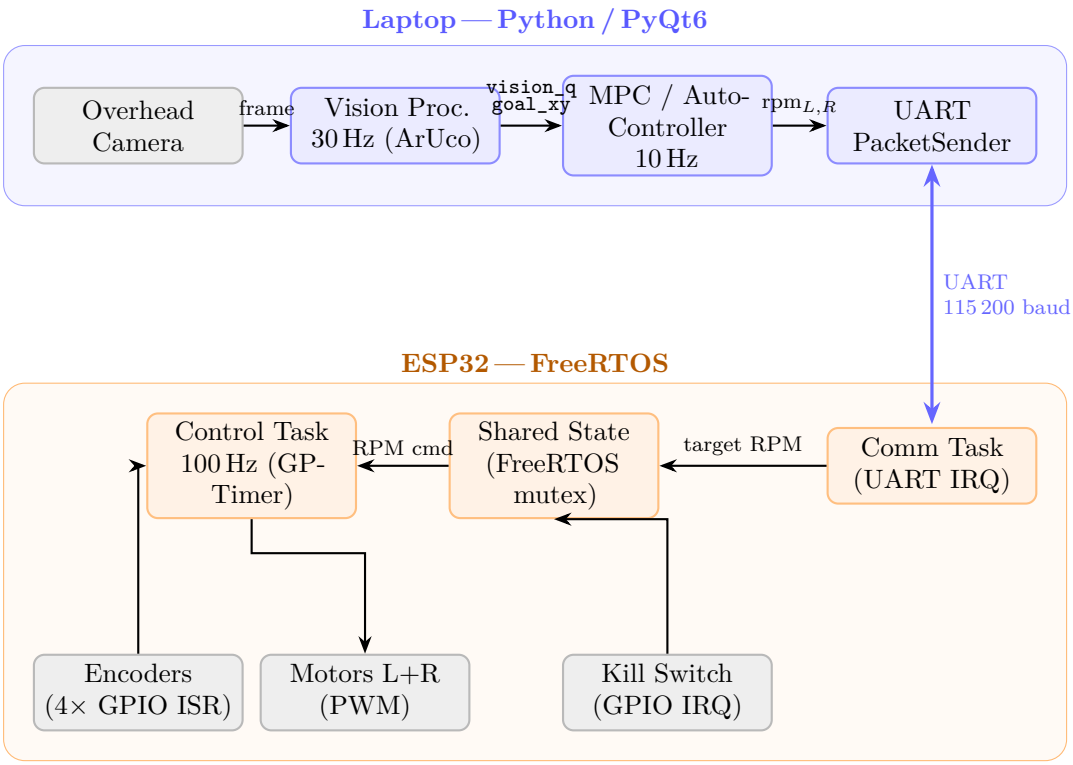

Figure 2: System architecture: data flow between the laptop and ESP32. Solid arrows show data flow; the double-headed arrow is the bidirectional UART link.

3 Highlight: Impressive Parts of the Code

3.1 Quadrature Encoder ISR

In order to read the encoders using interrupts, each encoder maintains its last 2-bit state. We use a 16-entry transition table that maps the concatenated old and new state (a 4-bit index) to either -1, 0, or $+1$ counts. An ISR fires on both the A and B edges, updates the state only on valid transitions, and accumulates counts using an atomic add — safe from any concurrent context.

```
static const int8_t transition_table[16] = {
    0, +1, -1, 0, -1, 0, 0, +1, +1, 0, 0, -1, 0, -1, +1, 0,
};

static void IRAM_ATTR encoder_gpio_isr(void *arg) {
  EncoderState *e = (EncoderState *)arg;
  const int a = gpio_get_level(e->pin_a);
  const int b = gpio_get_level(e->pin_b);

  const uint8_t new_state = (((a & 1) << 1) | (b & 1));
  const uint8_t index   = ((e->last_state << 2) | new_state);

  const int8_t step = transition_table[index];
  if (step != 0) {
    e->last_state = new_state;
    atomic_fetch_add(&e->pending, step);
  }
}
```

3.2 Control Timer

Motor control runs on a fixed period of 10 ms (100 Hz). A GPTimer auto-reloads at that interval; its alarm callback calls `vTaskNotifyGiveFromISR` to unblock the Control Task. The task blocks on `ulTaskNotifyTake`, then runs PID, reads targets/encoders, updates PWM, and sends telemetry. This notify-from-ISR pattern keeps the ISR itself minimal (no computation in interrupt context) while guaranteeing the task wakes at exactly the right time.

```
static bool IRAM_ATTR on_control_timer_alarm(
    gptimer_handle_t timer,
    const gptimer_alarm_event_data_t *edata,
    void *user_ctx) {

  TaskHandle_t control_task_handle = (TaskHandle_t)user_ctx;
  BaseType_t higher_priority_task_woken = pdFALSE;
  vTaskNotifyGiveFromISR(control_task_handle,
                         &higher_priority_task_woken);
  return higher_priority_task_woken == pdTRUE;
}
```

3.3 MPC

The high-level path planner is a nonlinear model predictive controller (NMPC) implemented in Pyomo [2] and solved with IPOPT [3]. At each decision step it optimizes a finite horizon of future motion. The state vector is:

$$\mathbf{x} = [x,\ y,\ \theta_t,\ \theta_l,\ v,\ \omega]^\top$$

where (x, y) is the truck rear-axle position, θ_t and θ_l are the truck and trailer headings, and v and ω are linear and angular velocity. The control inputs are linear acceleration a and angular acceleration α.

The dynamics constraints are a discrete-time Euler integration of the truck–trailer equations: the truck advances along its heading, headings integrate from angular velocity, the trailer heading evolves from the hitch geometry, and velocities integrate from the accelerations.

The optimization enforces hard limits on linear speed and turn rate, and a jackknife constraint that keeps the heading difference $|\theta_t - \theta_l| \leq \theta_{\max}$ so the configuration stays physically plausible.

The objective is a weighted sum of squared errors: it penalizes missing the desired pose at the end of the horizon, missing the goal at intermediate steps, and using too much control effort.

```
def _build_model(self) -> pyo.ConcreteModel:
    c = self.cfg
    m = pyo.ConcreteModel()
    m.T     = pyo.RangeSet(0, c.N)
    m.K     = pyo.RangeSet(0, c.N - 1)
    m.S     = pyo.RangeSet(0, 5)
    m.x     = pyo.Var(m.S, m.T, domain=pyo.Reals)
    m.a     = pyo.Var(m.K, bounds=(c.a_min,  c.a_max))
    m.alpha = pyo.Var(m.K, bounds=(c.alpha_min, c.alpha_max))

    def dyn_rule(model: Any, s: int, k: int):
        if s == 0:
            return (model.x[0, k+1] ==
                    model.x[0, k] + c.dt*model.x[4, k]*pyo.cos(model.x[2, k]))
        if s == 1:
            return (model.x[1, k+1] ==
                    model.x[1, k] + c.dt*model.x[4, k]*pyo.sin(model.x[2, k]))
        if s == 2:
            return (model.x[2, k+1] ==
                    model.x[2, k] + c.dt*model.x[5, k])
        if s == 3:
            return (model.x[3, k+1] ==
                    model.x[3, k] + c.dt*(model.x[4, k]/c.d)
                    *pyo.sin(model.x[2, k] - model.x[3, k]))
        if s == 4:
            return (model.x[4, k+1] ==
                    model.x[4, k] + c.dt*model.a[k])
        if s == 5:
            return (model.x[5, k+1] ==
                    model.x[5, k] + c.dt*model.alpha[k])

    m.dyn = pyo.Constraint(m.S, m.K, rule=dyn_rule)
    m.vel_bounds = pyo.Constraint(
        m.T, rule=lambda model, t:
        pyo.inequality(c.v_min, model.x[4, t], c.v_max))
    m.omega_bounds = pyo.Constraint(
        m.T, rule=lambda model, t:
        pyo.inequality(c.omega_min, model.x[5, t], c.omega_max))
    m.jackknife = pyo.Constraint(
        m.T, rule=lambda model, t:
        pyo.inequality(-c.max_jackknife_angle,
```

```
                    model.x[2, t] - model.x[3, t],
                    c.max_jackknife_angle))

    def obj_rule(model: Any):
        cost = (
            c.w_pos * ((model.x[0, c.N] - c.q_des[0])**2
                    + (model.x[1, c.N] - c.q_des[1])**2)
            + c.w_theta_t * (model.x[2, c.N] - c.q_des[2])**2
            + c.w_theta_l * (model.x[3, c.N] - c.q_des[3])**2
            + c.w_v      * (model.x[4, c.N] - c.q_des[4])**2
            + c.w_omega  * (model.x[5, c.N] - c.q_des[5])**2
        )
        cost += sum(
            c.w_pos_stage
            * ((model.x[0, k] - c.q_des[0])**2
             + (model.x[1, k] - c.q_des[1])**2)
            for k in range(c.N)
        )
        cost += sum(
            c.w_a * model.a[k]**2 + c.w_alpha * model.alpha[k]**2
            for k in range(c.N)
        )
        return cost

    m.obj = pyo.Objective(rule=obj_rule, sense=pyo.minimize)
    return m
```

4 Time Machine — What We Would Do Differently

If we could go back to the start of the project, we would do most of what we did over again. However, there are some improvements that would have been nice to add, and things we would do differently.

4.1 Hardware Jackknife Detection

Right now the jackknife constraint only lives in the MPC optimization and relies entirely on the vision-based state estimate. We do have a sensor on the hitch angle (the potentiometer), but its accuracy is lower than the vision-derived estimate due to the quality of the potentiometer. If we could go back, we would source a higher-quality angle sensor. This would allow us to use the hitch angle directly in the MPC instead of relying on the camera, and would also enable a hardware interrupt on the ESP32 that cuts motor power immediately if a jackknife condition is detected — faster and more reliable than waiting for the next MPC iteration.

4.2 Obstacle-Aware MPC

The current MPC has no awareness of obstacles other than the jackknife angle. In a real parking scenario the truck would need to avoid walls and other objects. An improvement would be to add obstacle constraints to the MPC formulation. The camera already has a full overhead view of the scene, so obstacle detection is feasible with the existing vision setup; however, this would increase the complexity and solve time of the MPC.

4.3 Parallel-Parking MPC

The current MPC drives the truck to park into a pre-determined spot regardless of the relative angles. In a real parking scenario the truck would want to park in parallel — that is, with the trailer aligned to the parking space at a specified heading. An improvement would be to add an additional angle state (trailer-to-parking-spot heading error) to the MPC state vector. The CV already detects all ArUco markers including the parking spot, so adding one more state is feasible with the existing vision setup; however, this would require changing cost functions and would potentially increase MPC complexity and solve time.

5 Conclusion

This project demonstrated a full-stack embedded control system for autonomous truck-trailer reverse parking. On the hardware side, an ESP32 running FreeRTOS achieved hard real-time motor control at 100 Hz using a GPTimer ISR, quadrature encoder interrupts, dual independent PID loops, and FreeRTOS mutex-protected shared state between the Control and Communication tasks. On the software side, a laptop-hosted Python stack combined an overhead ArUco vision pipeline (30 Hz), a nonlinear MPC planner (10 Hz, Pyomo/IPOPT), and a PyQt6 GUI into a single event-driven application. The system successfully performed closed-loop autonomous parking maneuvers on the physical model, with the MPC enforcing the jackknife constraint throughout the trajectory.

Building an embedded system that spans hardware, software, real-time control, computer vision, nonlinear optimization, and a GUI was a fun, challenging, and rewarding project.

5 References

[1] FreeRTOS.org. *FreeRTOS Kernel Developer Documentation*. https://www.freertos.org/Documentation/RTOS_book.html

[2] W. E. Hart, C. D. Laird, J.-P. Watson, D. L. Woodruff, G. A. Hackebeil, B. L. Nicholson, and J. D. Siirola. *Pyomo — Optimization Modeling in Python*, 2nd ed. Springer, 2017.

[3] A. Wächter and L. T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.

[4] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez. Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6):2280–2292, 2014.

[5] J.-P. Laumond, P. Jacobs, M. Taïx, and R. Murray. A motion planner for nonholonomic mobile robots. *IEEE Trans. Robotics and Automation*, 10(5):577–593, 1994.